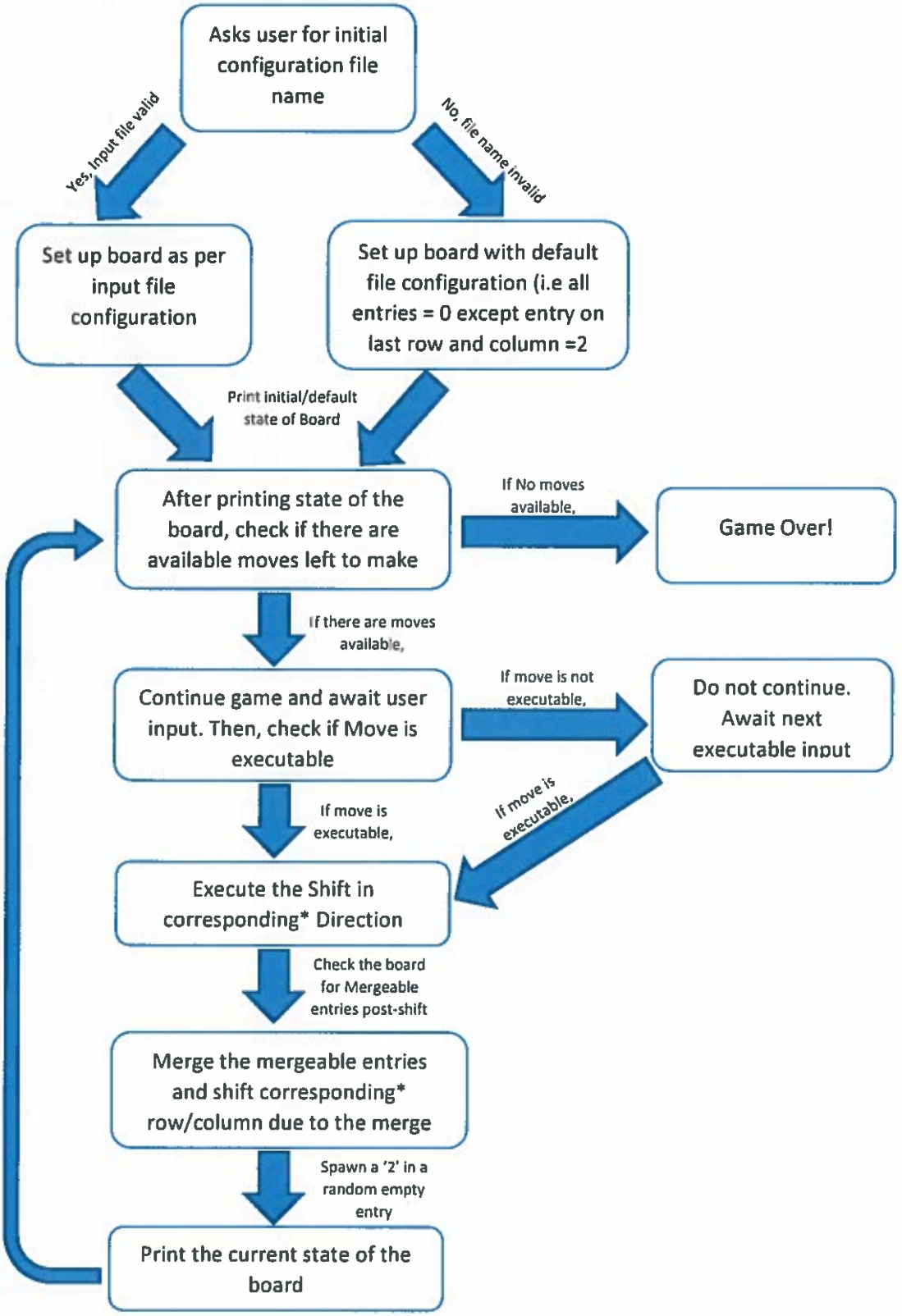
High end average 95

Spring Assignment -Programming the 2048 Game

Ying Hao Lua – CID no. 01044210

Considerations when designing the 2048 game:



Based on the considerations in the form of a flow chart above, I have designed the 2048 code such that the functions are modular and yet distinct. Basically, to take us from one level to the next, we have an arrow in the diagram representing a function to either request for Input/Output or do some checks or execute the move by the user.

I have chosen to use a 2 Dimensional Array in this project to code the 2048 game as I felt that it would make my code the most readable as well as efficient. This is because, we know that there the elements in the game is fixed with 4 by 4 cells, each containing a number that could at most go up to $2^{17} = 131072$. Thus, I also chose the data type in each element to be of type integer. This would allow us to minimise memory required and probably execute the program effieciently and effectively since a 2-Dimensional array is very similar to that of the 2048 Game Layout.

With reference to the flowchart which clearly breaksdown the tasks required to program a fully functional 2048 game, I will now briefly summarise the functions used.

The first task is to request the input file configuration name from the user. This is performed in the main and not called as a function since we only execute this once throughout the game. If Input configuration file name was found and valid, we set up the board as per the input file. If not, we call boardInitialise() and set up the board with the default configuration(All entries = 0 except for the entry on the bottom row and the rightmost column). Although function is only called once here, this makes the main looks neater and improves modularity.

Next, we print the Board with the Initial/Default configuration by calling the function printBoard(). Following which, we check this current state of the board to see if there are any possible moves available by calling the function availableMoves(). If there no available moves are left, game is over and we print out "Game Over!". If there are possible moves to be made, we continue by waiting for user to input the direction of move.

Here, we check if the input move results in any change to the state of the board. If the input direction was invalid or if no entries were changed from the resulting input move, then program should not continue and wait till an executable move comes by as per the program specifications. We call the function shiftBoard() which returns a boolean value of false should the move not result in any shift or change in the state of the board. If the return value was true, we know at least one element has changed or is shifted. This shiftBoard() function works by calling by calling a sub module(represented by " * " in the flowchart) based on the input direction. Depending on the input shift direction given by the user, shiftBoard() calls a sub module, i.e shiftBoardup() to perform the shift on the relevant entries then calls another sub module i.e mergeUp() to see if there are any adjacent entries that can be merged in the direction of the shift after the shift. If there are any, the sub module will merge them in the direction of the shift and shift the entries in that row or column correspondingly to fill up the void created by that merge.

In doing so, functional decomposition as encouraged by Dr Cattafi is better executed as each tasks becomes more technically specific in the task they are to perform. In addition, it prevents having to write all of the shift implementation under one function which would be really lengthy and upwards of 80 lines and thus improves readability of the code, leading to an overall improvement in the quality of the code.

Following this, we spawn a '2' in a random empty entry as per game specifications by calling the generateSpawn() function. We then call the printBoard() function again to print out the current state of the board containing all the entries. The program then checks if there are any available moves left and the game continues in this loop until the game is over (i.e no more possible moves to be made).